

AURA

Agentic Workspace & AI Orchestration Platform

Project Blueprint for Codex and A3 Delivery

Course: Projeto e Engenharia de Software

Academic objective: build a portfolio-grade open-source software project reverse-engineered from real market requirements.

Primary market reference: Apple - Sr Python Backend Engineer, AIML (Role 200609199-0836)

Complementary product reference: Apple - Swift Software Engineer - Agentic OS Experiences (Role 200643348-0836)

This document is the source of truth for project scope, architecture, stack, quality standards and implementation order.

1. Project Context and Goal

The semester assignment requires a complete open-source software project designed from the technical expectations of real Pleno/Senior job openings. The evaluation emphasizes professional engineering practices: architecture, automation, testing, documentation, collaboration and software quality - not only whether the application runs.

AURA is therefore designed as a portfolio-quality backend and agent platform that demonstrates software engineering skills aligned with modern AI infrastructure work.

Primary goal: create an agentic workspace platform that receives natural-language tasks, plans steps, selects tools, executes approved actions, persists execution history and exposes the process through a modern dashboard.

1.1 Market references

Reference	Role	Why it matters
Primary	Apple - Sr Python Backend Engineer, AIML Role 200609199-0836 Cupertino	Justifies Python backend, AI/ML infrastructure mindset, data-oriented services, reliability
Complementary	Apple - Swift Software Engineer - Agentic OS Exploration Role 200643348-0836 Cupertino	Inspires the agentic product concept: AI-driven device/workspace interaction, 0-to-3

1.2 Why the implementation is not Swift-first

AURA uses Python and TypeScript because the primary reference role is explicitly a Python backend position, while the product concept is informed by the Agentic OS role. The project is intentionally cross-platform and must be fully developable on Windows. Swift can remain a future optional client, not a core dependency.

2. Product Definition

AURA is an agentic workspace and AI orchestration platform. A user submits a task in natural language. The backend converts that request into an execution plan, chooses from registered tools, requests approval for sensitive actions, executes steps, records telemetry and returns a final result.

2.1 Example workflows

- "List every Python file in this workspace and group them by project."
- "Find the documents related to Software Engineering and summarize what is due."
- "Organize these documents into suggested folders, but ask before moving anything."
- "Analyze this folder and show which files were modified recently."

2.2 MVP - version 0.1

- Create a task from the dashboard.
- Send a natural-language instruction to an agent.
- Generate a structured execution plan.
- Select at least one registered tool.
- Execute the tool safely.
- Persist task, plan, tool call and result in PostgreSQL.
- Display execution status and operational events in the frontend.
- Return a final user-facing result.

2.3 Explicit non-goals for the MVP

- Do not build a real operating system.
- Do not implement unrestricted autonomous file deletion.
- Do not depend on a single AI provider.
- Do not add multiple programming languages without a concrete engineering need.
- Do not build multi-agent orchestration before the single-agent path is stable.
- Do not optimize for production-scale cloud deployment before local reliability and tests are complete.

3. Locked Technology Stack

The following stack is the default and should not be changed casually. Prefer mature, mainstream libraries with strong documentation and broad ecosystem support.

Layer	Technology	Purpose
Backend language	Python 3.13	Core services, agent engine, tools and integrations
API	FastAPI + Pydantic	REST API, validation, typing and OpenAPI
Frontend	TypeScript + React + Vite	Dashboard and task execution UI
Database	PostgreSQL	Persistent application and execution data
ORM / migrations	SQLAlchemy + Alembic	Database access and schema evolution
AI abstraction	Provider interface	Decouple agent logic from a specific model vendor
Local AI option	Ollama (optional)	Development/demo without mandatory paid API calls
Backend tests	pytest + pytest-cov	Unit/integration tests and coverage
Frontend tests	Vitest	Frontend unit/component tests
E2E	Playwright (phase 2)	Critical user flows in browser
Python quality	Ruff + mypy	Lint/format and static typing
Frontend quality	ESLint + Prettier	Lint and formatting
Containers	Docker Compose	Reproducible local environment
CI/CD	GitHub Actions	Automated checks for push and PR
Coverage	Codecov	Visible coverage reporting
License	MIT	Required open-source license

Engineering principle: boring technology beats novelty. Add complexity only when it solves a documented project requirement.

4. Target Architecture

AURA uses a modular monolith for the semester project: clear internal service boundaries without premature microservices. This keeps the system understandable, testable and deployable while still demonstrating architecture discipline.

4.1 Logical flow

```
User -> React UI -> FastAPI -> Task Service -> Agent Engine -> Planner -> Tool Registry -> Tool  
Executor -> PostgreSQL -> Operational Event Stream -> UI
```

4.2 Core modules

API layer: HTTP endpoints, request validation, auth boundary when added, error contracts.

Task Service: Creates tasks, controls lifecycle and coordinates execution.

Agent Engine: Converts user intent into plans and coordinates step execution.

Planner: Produces structured plans; output must be machine-readable and validated.

Tool Registry: Registers tools by name, description, parameters and permission level.

Tool Executor: Executes validated calls, captures errors, timing and results.

Approval Service: Blocks sensitive operations until explicit user confirmation.

Memory Service: Persists useful task/workspace context without coupling it to one LLM.

AI Provider Layer: Uniform interface for real providers, local models and deterministic mocks.

Observability: Stores structured events, durations, status and failure data.

4.3 AI provider contract

Define an internal interface such as `AIProvider.generate(messages, options)` and, only if needed later, `AIProvider.embed(input)`. The agent engine must depend on this interface, never directly on one vendor SDK.

Required implementations for development: **MockProvider** (deterministic tests), one real API provider, and optionally **OllamaProvider**.

5. Agent Tools, Permissions and Safety

Tools are explicit capabilities. The agent may only perform actions through registered tools. Each tool declares a schema, a permission level and a deterministic execution boundary.

Permission	Examples	Behavior
READ	File search, metadata, read text	May execute automatically within configured workspace
WRITE	Create note, generate output file	May require confirmation depending on scope
MOVE	Move or rename file	Requires explicit user approval
DELETE	Delete file	Always requires explicit approval; disabled by default for MVP

Initial tool set

- FileSearchTool
- FileReadTool
- WorkspaceListTool
- CalculatorTool
- DocumentSummaryTool
- NotesTool

Do not expose unrestricted shell execution to the agent in the MVP. Any future command tool must use an allowlist, workspace sandbox and explicit approvals.

6. Persistence and Observability

Suggested core tables

- users - minimal identity if authentication is introduced
- workspaces - user-approved working directories or logical workspaces
- tasks - user request and lifecycle status
- plans - structured plan generated for a task
- executions - run-level metadata
- tool_calls - tool name, arguments, timestamps, status and result summary
- approvals - requested sensitive action and approval outcome
- memories - persisted non-sensitive workspace/task context
- audit_logs - append-only operational records

Operational metrics visible in the dashboard

- Successful vs failed executions
- Average task latency
- Average tool latency
- Tool usage counts
- Recent errors
- Execution timeline per task

Do not expose hidden model chain-of-thought. The UI should show operational states and structured execution events only: planning, waiting approval, tool selected, executing, succeeded, failed.

7. Repository and Collaboration Standard

7.1 Monorepo structure

```
aura/
├── backend/
│   ├── app/{api,agents,core,models,services,tools,database}/
│   ├── tests/
│   └── frontend/
│       ├── src/
│       └── tests/
├── docs/
├── .github/{workflows,pull_request_template.md}/
├── docker-compose.yml
├── README.md
├── CONTRIBUTING.md
└── LICENSE
```

7.2 Git workflow

- Protected main branch; no direct feature development on main.
- Use develop if the team wants an integration branch; otherwise short-lived feature branches may target main through PRs.
- Branch examples: feature/agent-engine, feature/tool-registry, feature/dashboard, bugfix/task-timeout.
- Every feature enters through a Pull Request.
- PRs must include what changed, why, test evidence and known limitations.
- Team members should leave real asynchronous code-review comments.
- Use Conventional Commits: feat:, fix:, test:, docs:, refactor:, chore:.

7.3 README requirements

- Project overview and motivation
- Feature list
- Architecture diagram
- Screenshots/GIFs
- Local setup
- Environment variables
- API documentation link
- Testing instructions
- Contributing guide
- Roadmap
- License
- Build, coverage and license badges

8. Quality, Testing and CI/CD

CI runs on every push and pull request. A failing required check means the change is not ready to merge.

Recommended pipeline

```
Checkout -> Install backend -> Ruff -> mypy -> pytest + coverage -> Install frontend -> ESLint -> Vitest -> Build -> Upload coverage
```

Testing strategy

- Unit tests: planner parsing, tool registry, permission rules, services, utilities.
- Integration tests: API + database + mock AI provider.
- Contract tests: AI provider interface and tool schemas.
- Frontend tests: critical components and task status transitions.
- E2E (phase 2): create task -> execute deterministic mock -> display result.
- No test should require a paid AI API. Use MockProvider for deterministic CI.

Initial coverage target: 80% backend. This is a project target, not a substitute for meaningful assertions.

9. Delivery Roadmap

Phase	Deliverable	Exit criterion
0 - Foundation	Repo, docs, Docker, FastAPI skeleton, React skeleton, PostgreSQL	System can start locally; CI passes
1 - MVP vertical slice	Create task -> mock plan -> tool -> result -> persistence -> dashboard	One complete deterministic agent flow works end-to-end
2 - Real AI provider	Provider abstraction + one real provider + structured planning	Can switch Mock/Real provider without agent-engine changes
3 - Tooling & approvals	Multiple tools + permission model + approval UI	Sensitive action cannot execute without approval
4 - Memory & observability	Memory service + metrics + execution timeline	Task history and operational events are visible
5 - Portfolio polish	README, GIFs, diagrams, tests, Codecov, release tag	Repository is interview-ready

Rule: do not begin a later phase by sacrificing the acceptance criteria of an earlier one.

10. Codex Operating Instructions

Codex should treat this PDF as the project specification. When a prompt conflicts with this document, explicitly call out the conflict before changing architecture or scope.

1. Inspect the existing repository before editing. Reuse established patterns.
2. Work in the smallest coherent vertical slice. Avoid speculative abstractions.
3. Keep the locked stack unless a change is justified in writing.
4. Prefer standard libraries and mature dependencies. Do not add a package for trivial functionality.
5. All AI-dependent behavior must be testable with MockProvider.
6. Never place secrets in source control. Provide .env.example only.
7. Database schema changes require Alembic migrations.
8. Every new backend feature must include or update tests.
9. Every API endpoint must have typed request/response models and predictable error responses.
10. Do not expose unrestricted system access to agents.
11. Do not claim a task is complete unless relevant tests/builds pass.
12. Before a large refactor, explain the reason and the migration plan.
13. Keep README and docs synchronized with behavior.

Definition of Done

- Feature implemented end-to-end
- Tests added/updated and passing
- Lint/type checks passing
- No secrets committed
- Migration included if schema changed
- Error paths handled
- Docs updated if behavior/setup changed
- Ready for PR review

11. Semester Acceptance Checklist

The repository should satisfy both the product goals and the academic checklist.

- Professional README.md
- Build, coverage and license badges
- UI screenshots or GIFs
- MIT license
- Feature and bugfix branches
- Structured Pull Requests
- Real asynchronous code review comments
- Semantic/traceable commits
- CI/CD on push and PR
- Build tooling appropriate to the stack
- Automated test coverage visible through Codecov or equivalent
- A complete end-to-end agent workflow
- Architecture documentation
- Reproducible local setup via Docker Compose

Success criterion: someone reviewing the repository should understand what AURA does, run it locally, inspect the architecture, see automated quality checks and follow the history of professional collaboration.

End of source-of-truth specification.